Week 4 - Domain 2 extras (OpsWorks, automation with Lambda / SDK / Step Functions, CloudFormation Git sync and IaC generator)

DOP-C02 Domain 2 items that are in the exam guide but weren't covered by the other topics. Typical questions:

1. A config management answer mentions **OpsWorks** = know what it was and that it's **end of life** - the modern answer is **Systems Manager**.
2. **Automate a recurring or complex task** across many resources / accounts = **EventBridge Scheduler, Lambda (SDK), Step Functions, SSM Automation**.
3. **Deploy a CloudFormation stack straight from Git** without building a pipeline = **CloudFormation Git sync**.
4. **Turn manually created resources into a template** = **CloudFormation IaC generator** (+ resource import / `cdk migrate`).

Part 1 - AWS OpsWorks (legacy, end of life)

Item	Detail
What it was	Managed **Chef / Puppet** configuration management: **OpsWorks Stacks** (Chef recipes, layers), **OpsWorks for Chef Automate**, **OpsWorks for Puppet Enterprise**
Status	**All three reached end of life in 2024** - no new stacks; existing customers migrated
Stacks concepts	Stack -> **layers** (web, app, DB) -> instances; **lifecycle events**: **Setup**, **Configure**, **Deploy**, **Undeploy**, **Shutdown**; auto healing; time-based and load-based instances
Configure event	Runs on **all instances** whenever an instance **comes online or goes offline** (update config such as load balancer member lists)
Replacement	**Systems Manager**: **State Manager** associations (desired state), `AWS-ApplyChefRecipes` / `AWS-ApplyAnsiblePlaybooks` documents, **Application Manager**, Patch Manager, Distributor; AWS provided a migration path to Application Manager

Exam use: OpsWorks may still appear in options. For **new** designs, the answer is **Systems Manager** (State Manager + Chef / Ansible documents) or IaC. If a question explicitly describes an existing OpsWorks Stacks app that must update configuration when instances join or leave, the **Configure lifecycle event** is the OpsWorks feature.

Part 2 - Automation for complex tasks (Lambda, SDK, Step Functions, Scheduler)

Choosing the automation tool

Need	Best choice
Run something **on a schedule** (cron / rate / one-time, time zones)	**EventBridge Scheduler** (or EventBridge scheduled rule)
Simple scheduled AWS API call (stop instances, start a task)	**EventBridge Scheduler universal target** (calls the API directly, **no Lambda**)
React to an event (resource created, finding, state change)	**EventBridge rule** -> target
Short custom code with the **AWS SDK** (under 15 minutes)	**Lambda**
Long-running / multi-step workflow, retries, branching, human approval, parallel work	**Step Functions**
Operational runbook on AWS resources / instances with **approvals and rate control**	**SSM Automation**
Commands on many instances	**SSM Run Command / State Manager**
Stop / start dev instances on business hours	**Instance Scheduler on AWS** solution, or Scheduler + SSM Automation / API

EventBridge Scheduler

- **Cron, rate and one-time** schedules, **time zones** and daylight saving, **flexible time windows**.
- **Universal targets**: call **thousands of AWS API actions directly** (for example `ec2:StopInstances`, `ecs:RunTask`) plus templated targets (Lambda, Step Functions, SQS, SNS).
- **Retry policy** and **dead-letter queue**; **execution role** the scheduler assumes.
- Scales to millions of schedules (per-customer reminders, one-time jobs).

Lambda automation with the AWS SDK

Practice	Detail
Credentials	**Execution role** - never access keys in code; secrets from **Secrets Manager / Parameter Store**
Limits	**15-minute timeout**; payload limits; concurrency quotas - long jobs go to **Step Functions**
API behaviour	Handle **pagination** and **throttling** (SDK retries with exponential backoff); respect **service quotas**
Reliability	**Idempotent** code (events can repeat); async **on-failure destinations** / DLQ; reserved concurrency to protect downstream
Cross-account	Central automation account Lambda calls `sts:AssumeRole` into a role in each target account (trust policy for the automation role; **ExternalId** for third parties); list accounts with **Organizations** `ListAccounts` (from management or delegated admin)
Multi-Region	Loop over Regions or deploy per Region (StackSets)
Observability	CloudWatch Logs / metrics, X-Ray, Powertools for structured logs

Step Functions for complex automation

Feature	Use
Standard workflows	Long (up to **1 year**), **exactly-once**, full execution history - business / ops workflows, approvals
Express workflows	Short (up to **5 minutes**), very high volume, at-least-once - event processing

Feature	Use
AWS SDK integrations	Call **200+ services' APIs directly** from a state (no Lambda glue)
`.sync` **pattern**	Start a job and **wait for completion** (CodeBuild, ECS task, Glue, nested Step Functions)
`.waitForTaskToken`	**Callback / human approval** (send token by email, resume on approval)
Map / Distributed Map	**Parallel** processing; distributed map for **millions of items** (S3 objects, accounts list)
Retry / Catch	Built-in error handling and fallbacks
Triggers	EventBridge rules / Scheduler, API, SSM, pipelines

Example: "Every night, for every account in the org, snapshot tagged volumes, copy to the DR Region, and email a report; some steps take 30+ minutes" = **EventBridge Scheduler -> Step Functions** (Map over accounts, SDK integrations / `.sync`, Retry / Catch) with **cross-account roles**.

Part 3 - CloudFormation Git sync and IaC generator

Git sync (GitOps for stacks)

Item	Detail
What	**Link a stack to a Git repository and branch**; **on every push, CloudFormation updates the stack automatically**
Repos	GitHub, GitHub Enterprise, GitLab, Bitbucket through **CodeConnections**
Deployment file	YAML in the repo: `template-file-path`, `parameters`, `tags`
Permissions	**IAM role** for Git sync (+ stack service role)
Extras	Optional **pull request comments** with change summaries; sync status and events in the console
Limits	No build / test / approval stages - for those use **CodePipeline**

"Simplest way to deploy a template whenever it's merged to main, no pipeline to maintain" = **CloudFormation Git sync**.

IaC generator (template from existing resources)

Item	Detail
What	**Scan** the account's resources, **select resources** (and related resources), **generate a CloudFormation template**
Then	**Import** the resources into a new / existing stack (managed by CloudFormation from now on) or **download** the template; generate a **CDK app** with `cdk migrate --from-scan`
Use case	**Manually created ("ClickOps") resources** must be brought under IaC **without recreating them**
Related	**Resource import** (existing resources into a stack - needs DeletionPolicy and identifiers), `cdk import`, **drift detection** after import

Part 4 - Exam traps

Trap	Truth
Choose OpsWorks for a new config management design	**End of life** - **Systems Manager** State Manager / Chef / Ansible documents
OpsWorks event when instances join / leave	**Configure** lifecycle event (runs on all instances)
Lambda for a 2-hour job	**15-minute limit** - Step Functions (`.sync`, Map) or ECS / Batch
Lambda only to call one AWS API on a schedule	**EventBridge Scheduler universal target** (no code)
Hardcoded access keys in automation	**Execution role** + `sts:AssumeRole` for other accounts
Cross-account automation	Central role assumes **target-account roles** (trust policy, ExternalId for third parties)
Throttling errors in a large SDK loop	**Retries with exponential backoff**, pagination, concurrency control
Human approval in a workflow	**Step Functions** `.waitForTaskToken` or SSM `aws:approve`
Process millions of S3 objects in parallel	**Step Functions Distributed Map**
Exactly-once, long-running audit trail	**Standard** workflow (Express = short, high volume)
Deploy stack on every Git push without a pipeline	**CloudFormation Git sync**
Need tests and approvals before stack update	**CodePipeline** (Git sync has no stages)
Manually created resources into IaC without recreating	**IaC generator** + import / `cdk migrate --from-scan`
Nightly stop of dev instances	**Instance Scheduler** or Scheduler -> StopInstances / SSM Automation

Part 5 - Hands-on lab (about 45 minutes, very low cost)

Lab 1 - EventBridge Scheduler without Lambda

1. Launch a t3.micro tagged `Env=dev`.
2. **EventBridge Scheduler -> Create schedule**: one-time (5 minutes from now) or cron `0 20 * * ? *` with your time zone; target **All APIs -> EC2 -> StopInstances** with `{"InstanceIds":["i-..."]}`; execution role with `ec2:StopInstances`; retries + DLQ.
3. The instance stops at the scheduled time - no Lambda.

Lab 2 - Step Functions with SDK integration and approval

1. State machine (Standard): `ListBuckets` via **SDK integration** -> **Map** over buckets calling `GetBucketEncryption` with **Catch** for errors -> **Pass** summary.
2. Add a task with `.waitForTaskToken` (SNS email containing the token); resume with `aws stepfunctions send-task-success --task-token ...`.

Lab 3 - Cross-account Lambda (optional, two accounts)

1. Target account: role `AutomationTarget` trusting the automation account's Lambda role, with `ec2:DescribeInstances`.
2. Lambda: `sts.assume_role(RoleArn=..., RoleSessionName="automation")`, then call EC2 with the temporary credentials.

Lab 4 - Git sync and IaC generator

1. Repo with `template.yaml` (one SNS topic) and `deployment.yaml` (`template-file-path: template.yaml`); CloudFormation -> **Create stack -> Sync from Git** with a CodeConnections connection; push a change (topic display name) and watch the stack update.
2. Create an S3 bucket manually; CloudFormation -> **IaC generator -> Start scan** -> select the bucket -> **Generate template** -> **import** into a new stack. Run drift detection.

Cleanup: delete schedule, state machine, Lambda / roles, Git sync stack and connection, generated stack (bucket retained per DeletionPolicy - delete manually), instance.

Part 6 - Check yourself

Questions

1. What was OpsWorks, what's its status, and what replaces it?
2. Which OpsWorks lifecycle event runs on all instances when one joins or leaves?
3. Simplest way to stop dev instances every evening without code?
4. When do you use Step Functions instead of Lambda?
5. Standard vs Express workflows?
6. How do you add a human approval to an automated workflow?
7. How does central automation reach many accounts securely?
8. A large SDK loop fails with throttling errors - fix?
9. How do you process millions of S3 objects in parallel?
10. Deploy a stack automatically on each push to main without a pipeline?
11. When is Git sync not enough?
12. Manually created resources must be managed by CloudFormation without recreation - how?
13. How do you get a CDK app from existing resources?

Answers

1. Managed **Chef / Puppet** (Stacks, Chef Automate, Puppet Enterprise); **end of life (2024)**; **Systems Manager** (State Manager, Chef / Ansible documents, Application Manager).
2. **Configure**.
3. **EventBridge Scheduler** universal target **ec2:StopInstances** (or Instance Scheduler).
4. **Long (> 15 min), multi-step, retries, branching, parallel, approvals**.
5. **Standard**: up to 1 year, exactly-once, history; **Express**: up to 5 minutes, high volume, at-least-once.
6. **Step Functions** `.waitForTaskToken` or SSM Automation `aws:approve`.
7. Automation role **assumes roles in each target account** (`sts:AssumeRole`, trust policy, ExternalId for third parties); accounts from **Organizations ListAccounts**.
8. **Exponential backoff retries**, pagination, lower concurrency, check quotas.
9. **Step Functions Distributed Map**.
10. **CloudFormation Git sync** (CodeConnections + deployment file).
11. When you need **build, test or approval stages** - use **CodePipeline**.
12. **IaC generator** scan -> template -> **import** into a stack.
13. `cdk migrate --from-scan` (IaC generator scan).

Quick memory summary

- **OpsWorks** (Chef / Puppet, Stacks layers, lifecycle Setup / Configure / Deploy / Undeploy / Shutdown) = **end of life** -> **Systems Manager** State Manager + Chef / Ansible documents.
- **Automation**: **EventBridge Scheduler** (cron / rate / one-time, universal API targets, retries, DLQ), **Lambda** (SDK, 15 min, execution role, idempotent, backoff), **Step Functions** (Standard / Express, SDK integrations, `.sync`, `.waitForTaskToken`, Distributed Map, Retry / Catch), **SSM Automation** (runbooks, approvals); cross-account = **AssumeRole**.
- **CloudFormation Git sync** = stack updates on Git push (CodeConnections, deployment file, no stages).
- **IaC generator** = scan existing resources -> template -> import / `cdk migrate --from-scan`.